

RANSAC Bonus Assignment

Over-determined solution · Explicit line form $y = mx + b$ · Robust circle fitting

Sambath Piazza · 3D Computer Vision · October 2026

Based on the RANSAC line-fitting notebook by Tarlan Ahadli (projects by Prof. Hajder Levente).

1. Overview

The task was to modify the original RANSAC line-fitting notebook in three ways. Each modification was placed in its own notebook so that every change is isolated and easy to follow. The notebooks build on each other: notebook 2 keeps the refinement of notebook 1, and notebook 3 reuses the whole structure.

Notebook	Bonus task	Points
1_ransac_line_overdetermined.ipynb	Over-determined (least-squares) solution at the end of RANSAC	1
2_ransac_line_explicit.ipynb	Implicit line $ax + by + c = 0$ replaced by explicit form $y = mx + b$	1
3_ransac_circle.ipynb	Full pipeline adapted to robust circle fitting	3

1.1 RANSAC in one paragraph

Half of the 1000 synthetic points lie on the true model (with Gaussian noise, $\sigma = 10$ px) and half are uniformly random outliers in a 640×480 image. Ordinary least squares on all points fails because every outlier pulls on the model. RANSAC instead repeats a *guess-and-check* loop: draw the **minimal sample** (2 points for a line, 3 for a circle), build the model through them, count the points closer than a threshold (the **consensus set**, threshold $\sigma = 10$ px), and keep the model with the largest consensus. The number of iterations is updated adaptively from the best inlier ratio w found so far:

$$k = \lceil \ln(1 - p) / \ln(1 - w^s) \rceil \quad (p = 0.95 \text{ confidence, } s = \text{sample size})$$

1.2 A fix applied to all three notebooks

The original loop was `for _ in range(max_iterations)`. Python evaluates `range(2000)` only once, so lowering `max_iterations` inside the loop had no effect and RANSAC always ran all 2000 iterations (the printed values 256, 125, 10 were computed but never used). All three notebooks use a `while i < max_iterations` loop, so the adaptive stopping rule really works: for the line data RANSAC now stops after **23** iterations instead of 2000. The cost is that fewer samples are tried (354 inliers instead of the 362 found by the 2000-iteration loop on the same data); the least-squares refinement below compensates for this.

1.3 Common structure

Every notebook follows the same six steps, and only the model-specific parts change:

1. Synthetic data generation (inliers on the model + uniform outliers), seed 0 for reproducibility.
2. Model helpers: fit from the minimal sample, point-to-model distance, `iteration_number`.
3. RANSAC loop (model-independent; only the sample size and the two helper functions differ).

4. **Over-determined refinement** on all inliers.
5. Comparison of the RANSAC model and the refined model (RMS inlier distance, error against the known truth).
6. Plot: all points, inliers, true model, RANSAC model (red dashed), refined model (orange).

2. Notebook 1 — Over-determined solution (1 point)

2.1 Idea

RANSAC's winning line is built from only two noisy points. With $n \approx 350$ inliers we have n equations $a x_i + b y_i + c = 0$ for only two degrees of freedom (three parameters minus the constraint $a^2 + b^2 = 1$): the system is **over-determined**. No line satisfies all equations, so we minimise the sum of squared residuals. Because the line is normalised, each residual is a true perpendicular distance, so this is *total least squares* (orthogonal regression):

$$\min_{a,b,c} \sum_i (a x_i + b y_i + c)^2 \quad \text{subject to} \quad a^2 + b^2 = 1$$

2.2 Derivation

- **Intercept c.** Setting $\partial/\partial c = 0$ gives $c = -(a \bar{x} + b \bar{y})$: the optimal line always passes through the **centroid** $(\bar{x}, \bar{y})$ of the inliers.
- **Normal (a, b).** Substituting c back, the residuals become $a(x_i - \bar{x}) + b(y_i - \bar{y})$. With the centred points as rows of a matrix A ($n \times 2$) and $\mathbf{n} = (a, b)$, the cost is $\|\mathbf{A}\mathbf{n}\|^2 = \mathbf{n}^T(\mathbf{A}^T\mathbf{A})\mathbf{n}$ with $\|\mathbf{n}\| = 1$. The minimiser is the eigenvector of $\mathbf{A}^T\mathbf{A}$ with the **smallest eigenvalue**, i.e. the last right-singular vector of A ($\mathbf{Vt}[-1]$ in the SVD). Intuition: the normal is the direction in which the inliers spread the least (PCA).

```
def fit_line_least_squares(pts):
    centroid = pts.mean(axis=0)          # best line passes through the centroid
    A = pts - centroid                  # one row per inlier: (x_i - x_mean, y_i - y_mean)
    _, _, Vt = np.linalg.svd(A)
    a, b = Vt[-1]                      # direction of smallest spread = unit normal
    c = -(a * centroid[0] + b * centroid[1])
    return a, b, c
```

2.3 Results (seed 0, true direction angle 304°, slope −1.48)

Line	a	b	c	RMS inlier distance	Angle error vs. true line
RANSAC (2 points)	−0.834	−0.552	376.18	5.50 px	0.46°
Refined (354 inliers)	−0.833	−0.553	376.19	5.49 px	0.39°

The RMS distance is guaranteed not to increase (least squares minimises exactly this quantity on exactly these points). The gain is small in this run because RANSAC happened to sample two very well placed inliers, so the notebook also repeats the experiment on 20 random datasets: the mean angle error drops from **0.47° to 0.36°** ($\approx 23\%$). The signs of (a, b, c) differ from the original notebook only because the normal is defined up to sign; it is the same line.

Why only ≈ 350 of the 500 generated inliers are counted: the threshold (10 px) equals the noise σ , so only $\approx 68\%$ of the true points fall inside the ± 10 px band, plus a few outliers that land there by

chance. The RMS of ≈ 5.5 px is the spread of a Gaussian truncated at $\pm 1\sigma$ ($\approx 0.54 \sigma$), which confirms the fit is at the noise floor.

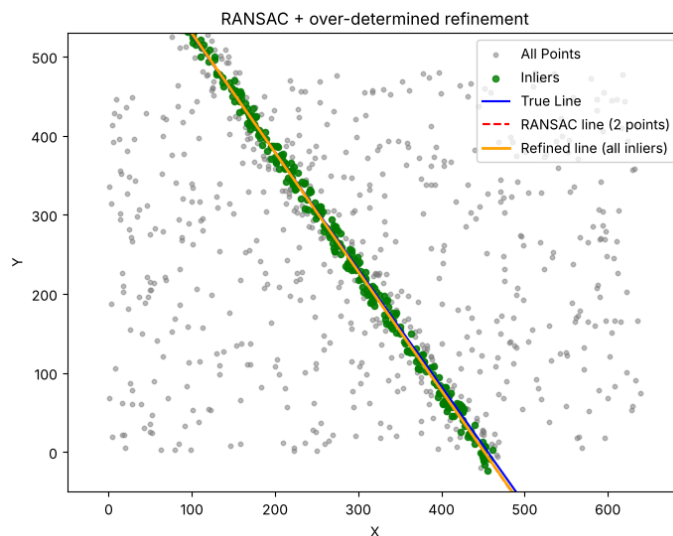


Figure 1 — Notebook 1. The red dashed RANSAC line is almost completely hidden under the orange refined line.

3. Notebook 2 — Explicit line form $y = mx + b$ (1 point)

3.1 What changes

The RANSAC loop only calls `fit_line_two_points` and `distance_point_line`, so it is untouched. Only the two helpers, the least-squares refinement and the plotting change:

Part	Implicit (original)	Explicit (notebook 2)
Parameters	(a, b, c) with $a^2 + b^2 = 1$	(m, b) : slope and intercept
Line through 2 points	normal $(-\Delta y, \Delta x) / \ \cdot\ $, $c = -(a x_1 + b y_1)$	$m = \Delta y / \Delta x$, $b = y_1 - m x_1$
Degenerate case	two identical points	$\Delta x = 0$ (vertical line) $\rightarrow$ None, sample skipped
Point distance	$ a x + b y + c $	$ m x - y + b / \sqrt{(m^2 + 1)}$
Least-squares fit	SVD of centred points	linear system $A \cdot [m, b]^T = y$, <code>np.linalg.lstsq</code>
Plotting	$y = (-a x - c)/b$	$y = m x + b$ directly

3.2 Details

- **Distance.** Writing $y = mx + b$ as $mx - y + b = 0$ gives the implicit form with $(a, b, c) = (m, -1, b)$, but $(m, -1)$ is not a unit vector, so the general formula $|ax + by + c| / \sqrt{(a^2 + b^2)}$ must be used. The **perpendicular** distance is kept (not the vertical $|y - (mx + b)|$), so $\text{sigma} = 10$ keeps the same meaning regardless of the slope.
- **Refinement.** Each inlier gives one equation that is linear in (m, b) : $m x_i + b = y_i$. Stacking rows $[x_i, 1]$ into A and solving $A \cdot [m, b]^T = y$ in the least-squares sense (normal equations, $(A^T A)^{-1} A^T y$) gives the refined line. Note this minimises *vertical* residuals, unlike the perpendicular residuals of notebook 1.

```
def fit_line_two_points(p1, p2):
    x1, y1 = p1; x2, y2 = p2
    if abs(x2 - x1) < 1e-8:          # vertical line: no finite slope
        return None
    m = (y2 - y1) / (x2 - x1)
    return m, y1 - m * x1

def distance_point_line(line_params, point):
    m, b = line_params; x, y = point
    return abs(m*x - y + b) / math.sqrt(m**2 + 1)

def fit_line_least_squares(pts):
    A = np.column_stack([pts[:, 0], np.ones(len(pts))]) # rows [x_i, 1]
    m, b = np.linalg.lstsq(A, pts[:, 1], rcond=None)[0]
    return m, b
```

3.3 Results (same data and seed as notebook 1)

Line	m (slope)	b (intercept)
True line	-1.483	675.92
RANSAC (2 points)	-1.509	681.07
Refined (354 inliers)	-1.502	679.03

RANSAC finds the same 354 inliers in 23 iterations as in notebook 1 (the representation does not change which points are inliers), and the refined line is closer to the truth in both parameters. The explicit parameters are easy to interpret and compare directly with the true slope d_y/d_x and intercept $p_y - m p_x$.

Trade-offs. Pros: only two parameters, direct interpretation, plain linear least squares, direct plotting. Cons: vertical lines cannot be represented, steep lines give huge slopes, and ordinary least squares minimises vertical rather than perpendicular error. This is why the implicit, normalised form is normally preferred in computer vision.

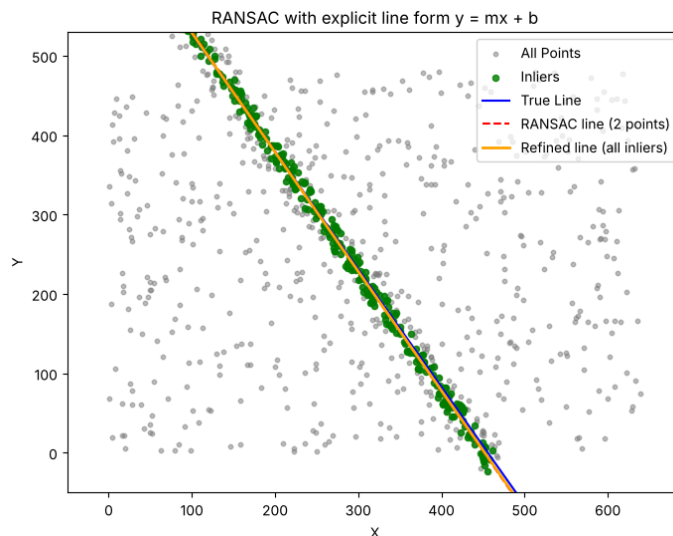


Figure 2 — Notebook 2, explicit form. Same data as Figure 1.

4. Notebook 3 — Robust circle fitting (3 points)

4.1 What changes

Part	Line	Circle
Parameters	(a, b, c)	centre (c_x, c_y), radius r
Minimal sample	2 points	3 points (iteration_number(..., 3, ...))
Synthetic inliers	$p + t \cdot d, t \in [-400, 400]$	$(c_x + r \cos \theta, c_y + r \sin \theta), \theta \in [0, 2\pi)$, same $N(0, 10)$ noise
Point distance	$ a x + b y + c $	$ \sqrt{(x - c_x)^2 + (y - c_y)^2} - r $
Degenerate sample	identical points	3 collinear points (matrix rank < 3) $\rightarrow$ None
Refinement	SVD / lstsq on all inliers	same fit_circle on all inliers

True circle: centre (320, 240), radius 150; 500 inliers + 500 outliers; seed 0.

4.2 One linear function for both the 3-point fit and the refinement

The circle equation $(x - c_x)^2 + (y - c_y)^2 = r^2$ is non-linear in its parameters, but expanding it gives

$$x^2 + y^2 + D x + E y + F = 0, \quad D = -2c_x, \quad E = -2c_y, \quad F = c_x^2 + c_y^2 - r^2$$

which is **linear in (D, E, F)**. Every point gives one equation $D x_i + E y_i + F = -(x_i^2 + y_i^2)$. With 3 points this is a 3×3 system (the exact circle through them, used inside RANSAC); with all inliers it is over-determined (the least-squares circle, used for the refinement). `np.linalg.lstsq` handles both, and the parameters are recovered as $c_x = -D/2$, $c_y = -E/2$, $r = \sqrt{c_x^2 + c_y^2 - F}$. If the three points are collinear the matrix has rank < 3 and the function returns None. The notebook checks the function on the points (7, 3), (2, 8), (-3, 3) $\rightarrow$ centre (2, 3), $r = 5$.

```
def fit_circle(pts):
    x, y = pts[:, 0], pts[:, 1]
    A = np.column_stack([x, y, np.ones(len(x))]) # rows [x_i, y_i, 1]
    rhs = -(x**2 + y**2)
    sol, _, rank, _ = np.linalg.lstsq(A, rhs, rcond=None)
    if rank < 3:
        return None
    D, E, F = sol
    cx, cy = -D / 2, -E / 2
    return cx, cy, math.sqrt(cx**2 + cy**2 - F)

def distance_point_circle(circle_params, point):
    cx, cy, r = circle_params; x, y = point
    return abs(math.sqrt((x - cx)**2 + (y - cy)**2) - r)
```

The least-squares step minimises the *algebraic* residual of the expanded equation rather than the exact geometric distance. For points spread around the whole circle, as here, the two agree closely, and the algebraic fit needs no iterations, so it is the simplest choice.

4.3 RANSAC changes

`ransac_circle` is `ransac_line` with three edits: `np.random.choice(len(points), 3, ...)`, the circle helpers instead of the line helpers, and sample size 3 in `iteration_number`. Because an all-inlier sample now has probability w^3 instead of w^2 , more iterations are needed: with $w = 0.352$, $k = \lceil \ln 0.05 / \ln(1 - 0.352^3) \rceil = 68$.

4.4 Results

Circle	Centre	Radius	RMS inlier distance
True	(320.0, 240.0)	150.0	—
RANSAC (3 points, 68 iterations)	(316.2, 239.4)	153.9	5.36 px
Refined (352 inliers)	(317.0, 240.2)	153.5	5.30 px

With 50 % outliers RANSAC recovers the circle to within a few pixels, and the refinement moves both the centre and the radius closer to the truth. Averaged over 20 random datasets (not in the notebook), the centre error drops from 6.2 px to 4.7 px and the radius error from 2.8 px to 2.2 px.

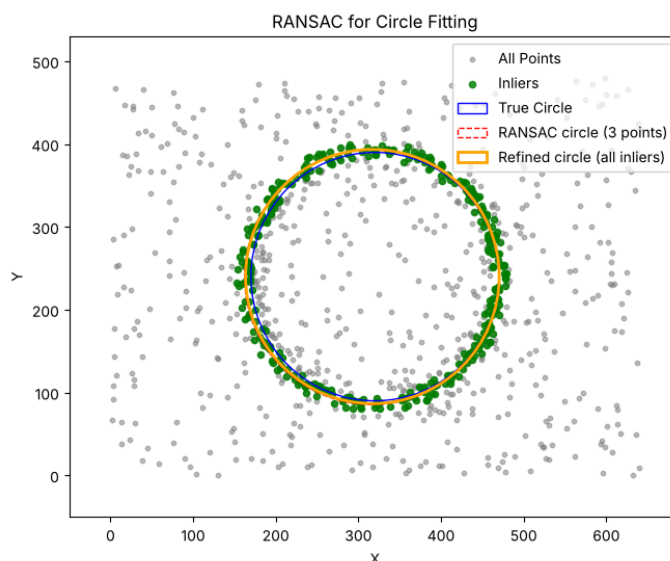


Figure 3 — Notebook 3. Blue: true circle; red dashed: RANSAC circle from 3 points; orange: least-squares circle from all inliers.

5. Summary

- **RANSAC decides which points to trust; least squares decides exactly where the model is.** The minimal sample makes RANSAC robust to outliers; the over-determined fit on the consensus set averages out the noise of the individual points.
- The RANSAC loop is model-independent: switching from implicit to explicit lines, or from lines to circles, only required changing the minimal-sample fit, the distance function and the sample size.
- Linearisation (circle: $x^2 + y^2 + Dx + Ey + F = 0$; explicit line: $m x + b = y$) lets one small `lstsq` call serve both the exact minimal fit and the over-determined refinement.
- The while-loop fix makes the adaptive iteration count of the original notebook actually take effect.

- Possible extension: re-select the inliers with the refined model and refit a few times. In tests this halves the circle-centre error again ($4.7 \text{ px} \rightarrow 2.4 \text{ px}$); it was left out to keep the notebooks minimal.